

Questions and Solutions: Multi-Head Attention (MHA)

Q1. Define Multi-Head Attention in one line.

Answer: Multi-Head Attention (MHA) runs several parallel scaled dot-product attentions (“heads”) and concatenates their outputs, allowing the model to attend to multiple relations simultaneously.

Q2. Why does the Transformer use multiple attention heads instead of one?

Answer: Multiple heads enable learning of different relationship types (semantic, syntactic, positional) in parallel, enriching the representation.

Q3. What are the main inputs to the attention mechanism?

Answer: Queries (Q), Keys (K), and Values (V).

Q4. Write the formula for Scaled Dot-Product Attention.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Q5. What is the purpose of dividing by $\sqrt{d_k}$?

Answer: It prevents very large dot-products that could saturate the softmax, keeping gradients stable.

Q6.

Find the output dimension when num_heads = 8 and value.dim = 64.

Answer: $8 \times 64 = 512$ before projection.

Q7. Name the four learnable projection matrices in MHA.

Answer: W_i^Q , W_i^K , W_i^V for each head, and shared output projection W^O .

Q8. State one advantage of Multi-Head Attention.

Answer: It captures multiple types of dependencies within the same sequence.

Q9. What is concatenated before applying W^O ?

Answer: The per-head outputs $\text{head}_1, \text{head}_2, \dots, \text{head}_h$.

Q10.

If num_heads = 4, how many independent attention computations occur?

Answer: 4 independent attentions.

Q11. Explain how Multi-Head Attention differs from Single-Head Attention.

Answer: Single-head computes one attention distribution; MHA projects inputs into multiple subspaces, applies attention independently in each, concatenates the results, and linearly mixes them, yielding richer contextual patterns.

Q12.

Describe the roles of Query, Key, and Value vectors.

Answer:

- **Query (Q):** Represents what information we seek.
- **Key (K):** Describes the available information at each position.
- **Value (V):** Contains the actual content to be aggregated based on attention weights.

Q13.

Why do we project Q, K, and V into lower dimensions?

Answer: To reduce per-head computation and memory while allowing each head to specialize in learning distinct relational subspaces.

Q14. Interpret each parameter in the Keras example below.

```
layers.MultiHeadAttention(  
    num_heads=4, key_dim=128, value_dim=64, output_shape=256)
```

Answer:

- `num_heads=4`: Four attention heads.
- `key_dim=128`: Dimension of queries and keys.
- `value_dim=64`: Dimension of each head's value/output.
- `output_shape=256`: Final output dimension after concatenation and linear projection.

Q15. Show how the final output dimension (256) is derived.

Answer: Each head produces 64-dimensional output; $4 \times 64 = 256$. The concatenated vector is linearly projected by W^O to match the required shape.

Q16.

Given `num_heads = 6` and `value_dim = 32`

(a) What is the concatenated vector length?

(b) If W^O maps to 128 dimensions, what is the output shape?

Answer:

(a) $6 \times 32 = 192$

(b) After $W^O : \mathbb{R}^{192} \rightarrow \mathbb{R}^{128}$, output dimension = 128.

Q17.

Explain what happens in $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

Answer: Inputs Q, K, V are projected by head-specific matrices. Each head performs scaled dot-product attention in its subspace, learning unique dependency patterns.

Q18. Describe the data flow in an MHA module.

Answer: Input embeddings $\rightarrow$ linear projections for Q, K, V per head $\rightarrow$ scaled dot-product attention $\rightarrow$ concatenation of head outputs $\rightarrow$ final projection $W^O \rightarrow$ output.

Q19. How do different heads learn different relationships?

Answer: Each head uses unique projection matrices, focusing on distinct feature subspaces; one head may attend to local syntax, another to long-distance dependencies, etc.

Q20. Write Keras code for MHA with 8 heads and 64-dimensional keys/values.

```
from tensorflow.keras import layers
mha = layers.MultiHeadAttention(
    num_heads=8,
    key_dim=64,
    value_dim=64,
    output_shape=512)
```

Answer: Creates 8 parallel heads (each 64-D). The concatenated vector (512-D) becomes the final output after projection.

Q21. What happens if `output_shape` is less than `num_heads × value_dim`?

Answer: Keras applies a learnable linear projection W^O to compress the concatenated vector to the desired smaller output size, losing some but not all information.

Q22. Modify to use 6 heads and output dimension 512.

```
mha = layers.MultiHeadAttention(
    num_heads=6,
    key_dim=64,
    value_dim=64,
    output_shape=512)
```

Answer: Each head produces 64-D ($6 \times 64 = 384$), then W^O expands it to 512-D output.

Q23. How does Keras internally handle concatenation and projection?

Answer: The layer computes all heads in parallel, concatenates along the last dimension, and applies a Dense layer (W^O) to reach `output_shape`. Users need not code these steps manually.